Praktyczne Aspekty Rozwoju Oprogramowania: TDD

Trainer(s):
Michał Orynicz (michal.orynicz@nokia.com)
Piotr Kowalczyk (piotr.1.kowalczyk@nokia.com)

Technikalia
Podstawowe pojcia
- Testy jednostkowe
- Dublerzy testowi

Test Driven Development
- Cechy
- Fazy
  - Cykl TDD
  - Faza czerwona
  - Faza zielona
  - Faza refaktoryzacji
- Wady i zalety TDD
- Podsumowanie

Praktyka
- Live coding
- Zadanie domowe

Zasady zajęć

- Na koniec udostępnione zostanie proste zadanie domowe do samodzielnego wykonania.
- Pomyślne wykonanie zadania domowego i wysłanie go do obydwu prowadzących da maksymalną ilość punktów za aktywność.

Podstawowe pojęcia

Podstawowe pojęcia

Pojęcia

- Test jednostkowy
- Iteracja
- Refactoring
- Testy regresyjne
- Pokrycie kodu
- Black box, white box
- Dublerzy testowi, mocki

Test jednostkowy

Co to jest UT?

- Testuje jednostkę kodu
- Kod który służy do weryfikacji czy kod produkcyjny działa poprawnie
- Pisany przez programistę implementującego funkcjonalność

Test jednostkowy

Jaki powinien być? - F.I.R.S.T.

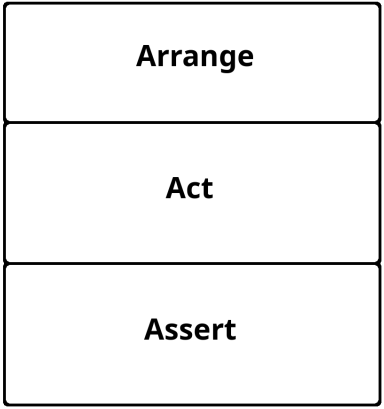
- Szybki (czas wykonania) (Fast)
- Niezależny od innych testów, środowiska (Independent)
- Powtarzalny, deterministyczny (Repeatable)
- Samoweryfikujący, dwustanowy (Self-validating)
- Dostarczony we właściwym momencie (Timely)

Test jednostkowy

Jaki powinien być? - c.d.

- Sprawdza co kod robi, a nie jak (funkcjonalny)
- Skupiony na jednej konkretnej rzeczy
- Czysty, czytelny
- Prosty - bez ifów, testuje tylko jeden scenariusz

Test jednostkowy



Test jednostkowy

Anatomia testu

- Test powinien być jasno podzielony na Przygotowanie (arrange), Testowaną akcję (act) i Weryfikację wyniku (assert)
- W szczególności nie chcemy mieć jednego testu w którym jest arrange, act, assert, znowu act, znowu assert....

Typy scenariuszy testów jednostkowych

- Scenariusze podstawowe
- Nieprawidłowy input
- Przypadki brzegowe

Dublerzy testowi

- Stub - Pusta implementacja lub zwracająca domyślną wartość, zatyczka
- Fake - Uproszczona implementacja w której można łatwo ustawić żeby zachowała się w konkretny sposób
- Spy - Jakaś implementacja zapisująca w jaki sposób została użyta
- Mock - Fake i spy w jednym

Test Driven Development

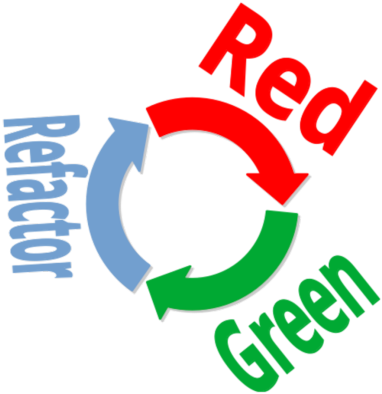
Test Driven Development

Czym jest TDD?

- Iteracyjna metoda dopisywania nowych funkcjonalności
- Programowanie "pod dyktando" testów

Fazy

Cykl TDD



Cykl TDD

3 Fazy

- Faza czerwona - piszemy test
- Faza zielona - zaliczamy testy kodem produkcyjnym
- Faza refaktoryzacji - czyścimy kod

Faza czerwona

Faza czerwona - scenariusz

- Wybieramy scenariusz testowy, który wymusi na nas napisanie najprostszej implementacji
- Scenariusze otwierające - wprowadzamy nową metodę, funkcję
- Scenariusze uszczegółowiające - doprecyzowują wymagane zachowanie w.w.
- Scenariusz wymyślamy zawsze z perspektywy użytkownika

Faza czerwona - test

- Czerwony test, to również taki który się nie kompiluje bo np nie ma jeszcze typu lub funkcji którą testuje
- Scenariusze testowe piszemy po jednym
- Test zawiera kompletny scenariusz

Faza czerwona - sens

- Zmusza nas do zdefiniowania konkretnych scenariuszy: konkretnych danych wejściowych i oczekiwanego wyniku
- Zapewnia, że funkcjonalność którą chcemy dopisać jeszcze nie działa
- Zmusza do przemyślenia interfejsu

Faza zielona

Faza zielona - implementacja

- Implementujemy tylko tyle kodu ile jest konieczne do zazielenienia **wszystkich** testów
- Kod na brudno, ścieżką najmniejszego oporu
- **Nie wolno** ruszać kodu testów

Faza zielona - sens

- Skupienie na wymaganiach a nie na jakości implementacji
- Szybki prototyp który pozwoli zweryfikować założenia z testu
- Punkt odniesienia dla refaktoryzacji

Faza refaktoryzacji

Faza refaktoryzacji - implementacja

- Refaktoryzujemy zarówno kod testów jak i kod produkcyjny
- Albo kod produkcyjny albo testy - nie na raz.
  Wyjątek: zmiana interfejsu
- Refaktoryzujemy "do skutku" - do wyczerpania pomysłów
- Nie zmieniamy zachowania względem istniejących testów
- Na koniec wszystkie testy są nadal zielone

Faza refaktoryzacji - sens

- Skupienie wyłącznie na jakości kodu (architektura, czytelność, utrzymywalność...)
- Łatwiej zauważyć wzorce i nazwać rzeczy które już istnieją
- Kod często refaktoryzowany jest bardziej plastyczny
- W razie niepowodzenia refaktoryzacji zawsze można wrócić do kodu z fazy zielonej i spróbować czegoś innego

Wady i zalety TDD

Zalety

- Wymusza zdefiniowanie wymagań przed pisaniem kodu
- Pozwala na bezpieczny refactoring
- Automatyczne testy regresyjne w zestawie
- Testy dokumentują przykłady użycia interfejsu
- Gwarantuje, że wszystkie funkcjonalności dla których mamy testy nie przestaną działać niepostrzeżenie
- Wykrywanie błędów często zanim się pojawią
- Pomaga pisać testy o wysokiej jakości

Wady testów jednostkowych

- 2 razy więcej kodu do napisania i utrzymywania
- Jeśli się zmieni interfejs, to trzeba poprawić wszystkie miejsca użycia
- Dodatkowy narzut na początkową konfigurację frameworku testowego

Wady TDD

- Źle wybrana kolejność scenariuszy może prowadzić do implementacji dużych ilości kodu na raz
- Nie ma gwarancji, że to co nie jest przetestowane działa
- Implementacja trwa dłużej

Podsumowanie

Podsumowanie

TDD

- Testy pisane przed kodem produkcyjnym
- Podejście iteracyjne złożone z jak najkrótszych kroków, w ramach określonych faz
- Dobre testy jednostkowe pozwalają na bezpieczną i częstą refaktoryzację

Podsumowanie

Testy jednostkowe

- Testy weryfikują funkcjionalność a nie sposób wykonania
- Testy powinny także być czyste i wymagają utrzymywania
- Scope UTków jest ograniczony, i nie zastępują one innych rodzajów testów, lecz je uzupełniają
- Testowanie więcej niż jednego realnego bytu w UTku nie jest błędem!

Live coding

Gra w życie

Podstawowe założenia

- Gra odbywa się na dwuwymiarowej planszy, na której pola nazywamy komórkami
- Komórki mają dwa stany: żywa i martwa
- W każdej turze wyliczamy nowy stan każdej komórki na podstawie stanu poprzedniego jej i jej 8 sąsiadów

Gra w życie

Zasady przejścia ze stanu żywej komórki

- Jeżeli komórka jest żywa i ma mniej niż 2 żywych sąsiadów, umiera z samotności
- Jeżeli komórka jest żywa i ma 2 lub 3 żywych sąsiadów, pozostaje żywa
- Jeżeli komórka ma więcej niż 3 żywych sąsiadów, umiera z przeludnienia

Gra w życie

Zasady przejścia ze stanu martwej komórki

- Jeżeli komórka jest martwa i ma dokładnie 3 żywych sąsiadów, zostaje powołana do życia
- W pozostałych przypadkach komórka pozostaje martwa

Zadanie domowe - kasa fiskalna

Kasa fiskalna - kryteria oceny

- Wymagane funkcjonalności działają prawidłowo
- Scenariusze pokrywają wszystkie oczekiwane przypadki
- Testy wyglądają zgodnie z wytycznymi
- Printowanie rzeczy nas nie interesuje - można printować co się chce, dla nas istotny jest wynik zwracany przez kod
- **Ważne**: Kod musi być oparty na naszym repozytorium, a wysłane paczki (np. .zip'y) nie zawierają binarek

Kasa fiskalna - wymagania funkcjonalne

- Dodawanie i usuwanie towaru do/z rejestru (id, nazwa, cena)
- Sprawdzenie wartości aktywnego rachunku
- Dodawanie i usuwanie towaru z rachunku
- Zwrócenie listy zarejestrowanych towarów w rachunku
- Możliwość zamknięcia rachunku i rozpoczęcia nowego

Kasa fiskalna - dodatkowe wymagania - dla chętnych

- Aktywowanie i deaktywowanie promocji w rejestrze
- Rejestrowanie lub usuwanie towaru do/z typu promocji
- Wylistowanie zarejestrowanych promocji
- Aktywacja promocji kartą stałego klienta skanowaną tak samo jak towary
  - Globalna np. 10% zniżki na wszystko
  - Na konkretne towary np. 3 sztuka gratis
- Poprawna obsługa towarów w promocji

Kasa fiskalna - wysyłanie zadania

- Zadania powinny być wysłane na oba adresy:
  1. piotr.1.kowalczyk@nokia.com
  2. michal.orynicz@nokia.com
- Paczka musi być oparta na repozytorium z prezentacji
- Paczka powinna być zawarta w archiwum, najlepiej .zip, bez binarek
- W treści maila proszę podać imię, nazwisko i numer indeksu
- Deadline podany zostanie na zajęciach oraz w mailu po

Kasa fiskalna - feedback

- Po sprawdzeniu otrzymacie zwrotkę w komentarzem i feedbackiem
- Feedback jest czysto informacyjny i ma na celu danie wskazówek co można było zrobić inaczej

Informacje techniczne

- Używany framework do testów - **GTest**;
- Repozytorium z programem startowym:
  `git clone https://github.com/nokmory/paro-tdd.git`
- Przykładowe użycie GTesta pokazane jest w pliku `src/sampleTests.cpp` w repozytorium.

Informacje techniczne

- Można korzystać z własnego środowiska, ale potrzebny jest zainstalowany **CMake** i **G++**.
- Aby uruchomić testy należy wykonać polecenia w terminalu:
  1. `cd paro-tdd`
  2. `mkdir build && cd build`
  3. `cmake ../workspace`
     Wystarczy tylko przed pierwszą kompilacją
  4. `make run_tests`
     Kompilacja + uruchomienie testów

Informacje techniczne

- Po dodaniu nowego pliku .cpp do projektu, należy dopisać jego nazwę w `CMakeLists.txt` w sekcji `add_executable`.
- Oczekujemy że wysłane repozytorium kompiluje się, wszystkie dopisane testy są uruchamiane i przechodzą